# 32.5 SOUL 安全

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~295k tokens，输出 ~5k tokens（本节）

SOUL.md 是 OpenClaw Agent 的"灵魂文件"——它定义了 Agent 的人格、行为准则和系统提示词。正因为 SOUL.md 处于信任链的最顶端，对它的篡改将产生最深远的安全影响。OpenClaw 内置了一个名为 `soul-evil` 的机制，既可作为安全研究工具，也可作为"Agent 人格测试"的趣味功能。

## 32.5.1 恶意 SOUL 检测（`src/hooks/soul-evil.ts`）

### 机制概述

`soul-evil` 是一个内置钩子，它可以在特定条件下将 SOUL.md 的内容替换为另一个文件——`SOUL_EVIL.md`。这个机制的设计意图有两方面：

1. **安全测试**：让开发者体验"SOUL 被劫持"的效果，从而更好地理解系统提示词安全的重要性
2. **趣味功能**：定期或随机切换 Agent 人格（如在特定时间段内让 Agent 变得"调皮"）

## 配置结构

```typescript
// src/hooks/soul-evil.ts

export type SoulEvilConfig = {
  /** SOUL_EVIL 文件名（默认：SOUL_EVIL.md） */
  file?: string;
  /** 随机触发概率（0-1） */
  chance?: number;
  /** 每日固定时间窗口 */
  purge?: {
    at?: string;          // 开始时间（HH:mm 格式）
    duration?: string;    // 持续时长（如 "30m", "1h"）
  };
};
```

## 触发判定

`decideSoulEvil` 函数根据两种条件判定是否使用恶意 SOUL：

```ts
// src/hooks/soul-evil.ts（简化）

export function decideSoulEvil(params: SoulEvilCheckParams): SoulEvilDecision {
  const fileName = evil?.file?.trim() || "SOUL_EVIL.md";
  if (!evil) return { useEvil: false, fileName };

  // 条件 1：每日净化窗口
  const timeZone = resolveUserTimezone(params.userTimezone);
  const inPurge = isWithinDailyPurgeWindow({
    at: evil.purge?.at,
    duration: evil.purge?.duration,
    now: params.now ?? new Date(),
    timeZone,
  });
  if (inPurge) {
    return { useEvil: true, reason: "purge", fileName };
  }

  // 条件 2：随机概率
  const chance = clampChance(evil.chance);  // 限制在 [0, 1]
  if (chance > 0) {
    const random = params.random ?? Math.random;
    if (random() < chance) {
      return { useEvil: true, reason: "chance", fileName };
    }
  }

  return { useEvil: false, fileName };
}
```

## 每日净化窗口

"净化窗口（Purge Window）"允许配置一个每天固定的时间段，在此期间使用 SOUL_EVIL.md。时间计算考虑了用户时区：

```typescript
// src/hooks/soul-evil.ts（简化）

function isWithinDailyPurgeWindow(params: {
  at?: string;          // 如 "14:00"
  duration?: string;    // 如 "30m"
  now: Date;
  timeZone: string;     // 如 "Asia/Shanghai"
}): boolean {
  const startMinutes = parsePurgeAt(params.at);  // "14:00" → 840
  const durationMs = parseDurationMs(params.duration);

  // 将当前时间转换为用户时区的毫秒偏移
  const nowMs = timeOfDayMsInTimezone(params.now, params.timeZone);

  const startMs = startMinutes * 60 * 1000;
  const endMs = startMs + durationMs;

  // 处理跨午夜的情况
  if (endMs < 24 * 60 * 60 * 1000) {
    return nowMs >= startMs && nowMs < endMs;
  }
  const wrappedEnd = endMs % (24 * 60 * 60 * 1000);
  return nowMs >= startMs || nowMs < wrappedEnd;
}
```

> **衍生解释**： `Intl.DateTimeFormat` 是 ECMAScript 国际化 API（ECMA-402）的一部分，可以将日期格式化为特定时区的表示。OpenClaw 使用 `formatToParts` 方法提取用户时区下的小时、分钟、秒，避免手动处理复杂的时区偏移计算（如夏令时切换）。

## SOUL 替换执行

`applySoulEvilOverride` 执行实际的 SOUL 文件替换。它在 Agent 引导（bootstrap）阶段被调用：

```typescript
// src/hooks/soul-evil.ts（简化）

export async function applySoulEvilOverride(params: {
  files: WorkspaceBootstrapFile[];
  workspaceDir: string;
  config?: SoulEvilConfig;
}): Promise<WorkspaceBootstrapFile[]> {
  const decision = decideSoulEvil({ config: params.config, ... });
  if (!decision.useEvil) return params.files;

  // 读取 SOUL_EVIL.md 内容
  const evilPath = path.join(workspaceDir, decision.fileName);
  let evilContent: string;
  try {
    evilContent = await fs.readFile(evilPath, "utf-8");
  } catch {
    log.warn(`SOUL_EVIL active but file missing: ${evilPath}`);
    return params.files;  // 文件不存在 → 安全失败
  }

  if (!evilContent.trim()) {
    log.warn(`SOUL_EVIL active but file empty: ${evilPath}`);
    return params.files;  // 文件为空 → 安全失败
  }

  // 确认 SOUL.md 存在于引导文件中
  const hasSoulEntry = params.files.some((f) => f.name === "SOUL.md");
  if (!hasSoulEntry) {
    log.warn("SOUL_EVIL active but SOUL.md not in bootstrap files");
    return params.files;
  }

  // 替换 SOUL.md 的内容为 SOUL_EVIL.md 的内容
  return params.files.map((file) => {
    if (file.name !== "SOUL.md") return file;
    return { ...file, content: evilContent, missing: false };
  });
}
```

## 安全失败设计

注意函数中的多重安全检查——如果 SOUL_EVIL.md 不存在、为空、或 SOUL.md 不在引导文件列表中，函数都会静默回退到正常行为（返回原始文件列表）。这种"安全失败（Fail-Safe）"设计确保即使配置错误也不会破坏 Agent 的正常启动。

## 配置解析

`resolveSoulEvilConfigFromHook` 从钩子配置中提取 soul-evil 参数，带有完整的类型验证和警告日志：

```
// src/hooks/soul-evil.ts（简化）

export function resolveSoulEvilConfigFromHook(
  entry: Record<string, unknown> | undefined,
  log?: SoulEvilLog,
): SoulEvilConfig | null {
  if (!entry) return null;

  const file = typeof entry.file === "string" ? entry.file : undefined;
  if (entry.file !== undefined && !file) {
    log?.warn?.("soul-evil config: file must be a string");
  }

  let chance: number | undefined;
  if (entry.chance !== undefined) {
    if (typeof entry.chance === "number" && Number.isFinite(entry.chance)) {
      chance = entry.chance;
    } else {
      log?.warn?.("soul-evil config: chance must be a number");
    }
  }

  // ... purge 配置解析类似

  if (!file && chance === undefined && !purge) return null;
  return { file, chance, purge };
}
```

## 安全启示

虽然 `soul-evil` 本身是一个"趣味"功能，但它揭示了 SOUL.md 安全的重要性：

1. **SOUL.md 是信任根**：Agent 的所有行为最终都受 SOUL.md 约束。如果 SOUL.md 被篡改，Agent 的行为将完全失控——甚至可能被武器化。
2. **工作区安全**：SOUL.md 存储在工作区目录中。确保工作区目录的文件权限（0o700）可以防止其他用户修改它。
3. **监控与审计**：`soul-evil` 的触发会记录调试日志。在生产环境中，监控这些日志可以发现异常的 SOUL 替换行为。
4. **最小权限**：`soul-evil` 只能替换 SOUL.md 的内容，不能修改其他引导文件——这是最小权限原则的体现。

# 本节小结

1. **`soul-evil`** 是一个内置钩子，可以在特定条件下将 SOUL.md 替换为 SOUL_EVIL.md，用于安全测试或趣味人格切换。
2. **两种触发条件**：每日净化窗口（固定时间段）和随机概率（0-1 之间的浮点数）。
3. **时区感知**：净化窗口使用 `Intl.DateTimeFormat` 进行用户时区的时间计算，支持跨午夜场景。
4. **安全失败设计**：文件不存在、为空、或 SOUL.md 缺失时，静默回退到正常行为。
5. **信任根保护**：SOUL.md 位于信任链顶端，其安全性依赖于工作区目录权限和文件系统审计。